Linux Capstone OSS project

Submitted by:
Himanshu Kothari
24BSA10245

A1. The Problem Linux Was Created to Solve

Before Linux came along, the world of operating systems was a choice between two worlds. You either used Unix, which was great but closed and pricey, or you played with Minix, which was supposed to be free but really wasn't. It didn't really work either way if you wanted a free operating system to mess with, learn from, and develop.

Unix was born back in the 1960s at Bell Labs. It basically changed the rules. Multitasking and a file system structure everyone uses today were born. But the problem was, it wasn't free. You couldn't see the source code, you couldn't modify it, and if you wanted to share it, you had to ask permission and pay for it. To any student or anyone who simply liked to tinker, it was absolutely out of the question.

Then there was Minix. It was supposed to be a free alternative. It came out in the '80s, mostly for teaching. You could see the source code, but you couldn't use it for business. It was a very simple system. If you wanted to use it for anything more than a toy, forget it.

"So where did Linux actually come in? Well, in 1991, Linus Torvalds, then just 21 years old and studying in Helsinki, got fed up of this dead end. He wanted a free Unix-like OS to run on his own 386 PC. No such animal existed, so he thought to himself: 'Why not me?' That August, he posted an offhand message on the comp.os.minix newsgroup:"

This is the pitch Linus Torvalds gave his OS, and how wrong it turned out to be. Today, we rely on it to run pretty much everything: web servers, Android smartphones, giant supercomputers, cloud computing, and even NASA's Mars rovers.

But the key to success was that Linus Torvalds shared his code. By linking his kernel to the GNU tools that Richard Stallman's team had been developing, suddenly the world had a fully functional, free OS: open source and ready to roll. And now we just call it Linux.

A2. The License — What It Actually Says

When Linus Torvalds first put Linux out in 1991, he created his own license for it. It was a relatively simple read, to say the least. But then, a year later, he chose to use the GNU General Public License version 2 for Linux. And this license, there were a lot more thoughts put into. And it's a huge reason why Linux has been able to succeed.

The GPL version 2 has something called 'copyleft.' We can use it, view how it works, change it, and share it with anybody we want to.
But then there is a catch. If we change it and want to share it, we have to share it under the same license. It is based on four main freedoms:

Freedom	What it means
Freedom 0	Run the program for any purpose — personal, work, etc…
Freedom 1	Study how the program works; access to source code is required
Freedom 2	Redistribute copies to others freely
Freedom 3	Distribute modified versions so the whole community benefits

GPL vs MIT — the key difference

	GPL v2 (Copyleft)	MIT (Permissive)
Share source?	Yes — always required if you distribute	No — you can keep changes private
Derivative licence	Must stay GPL	Can use any license
Commercial use	Allowed, but source stays open	Allowed, no conditions
Who uses it	Linux, Git, MySQL	React, jQuery, VS Code

"If I wanted to create a small library, and I wanted others to use it without hassle, I would choose the MIT license. For larger projects, where I want to ensure that upgrades and changes are free, I would choose GPL. That's exactly what Linux did, and honestly, it's the right thing to do considering what Linux is meant for."

People like Stallman always talk about "free beer" versus "free speech." "Free beer" means it's free, i.e., you don't pay for it. "Free speech" means not only is it free, but it's free to use, modify, and share.

A3. The Ethics of Open Source

Should all software be open source? This is a tricky one. We do owe a lot to open code: Apache, Linux, Git, Python – they're the backbone of the internet because someone decided to share, not hoard, their code. However, good code takes time and money to make. Without a way to protect that time and money, there's a lot of code that would not be made.

Is it fair to make money off community-created code?
From a legal standpoint: absolutely. GPL licenses make it perfectly legal to make money off other people's code. Red Hat, Google, Meta – they've all done it with Apache. However, it gets tricky on an ethical standpoint when we're using other people's free labor extensively – it's nice to give back in some way, whether it's code or money to the project. Of course, not all companies do this.

For installation:

Debian and Ubuntu, it's managed by apt.
RHEL and Fedora, it's managed by dnf.

And of course, we could just download the sources from kernel.org and build it ourselves. Most people will just stick to their distribution's version. It's easier and faster.

```
$ sudo apt update && sudo apt install linux-image-
 generic $ uname -r
```

Key Directories

Directory	What lives there
/boot	Kernel image (vmlinuz), initrd, GRUB config
/lib/modules/$(uname -r)	Loadable kernel modules for the running version
/proc	Virtual filesystem — live kernel and process info
/sys	Virtual filesystem — hardware and driver info
/var/log/kern.log	Kernel log messages (Debian-based systems)

User, Security and Service Management:

The kernel is not another program; it is at the heart of the system, not running as an ordinary user space process. When threads are running, they are running as root (UID 0), so if there is a bug in the kernel, it is not only a pain, it is a security risk.

We have tools such as lsmod, modprobe, rmmod, which allow us to load or remove kernel modules. If we need to change system settings dynamically, sysctl is used. For security patches, the package manager is used.

Dependencies

Dependency	Purpose
GCC / Clang	Compilers used to build the kernel from source
GNU Make	Build system that processes the kernel Make files
GNU Binutils	Assembler and linker used during compilation
glibc	Standard C library — user space programs use this to talk to the kernel
OpenSSL	Used for kernel module signing and secure boot

What Linux Has Enabled

Project	Relationship to Linux
Android	Built on the Linux kernel — billions of devices worldwide
Git	Linus built Git in 2005 because no good open DVCS existed for Linux dev
Docker	Linux kernel namespaces and cgroups are the foundation of containers
AWS / GCP / Azure	All major cloud platforms run on Linux under the hood
ChromeOS	Google's desktop OS built on the Linux kernel

Community and the LAMP Stack

The 'L' in LAMP stands for Linux. The traditional LAMP stack, consisting of Linux, Apache, MySQL, and PHP, has been the backbone of web hosting for more than two decades. The kernel is managed by Linus Torvalds and his trusted team through the Linux Kernel Mailing List. The Linux Foundation, with the support of Intel, Google, Microsoft, IBM, etc., provides organizational support. New versions are released every 9-10 weeks.

Dimension	Linux (Open Source)	Windows Server (Proprietary)
Cost	Free — no licensefee	Expensive per-server licensing
Security auditing	Anyone can review the code for bugs	Black box — trust Microsoft's word
Support	Community + paid options (Red Hat,	Microsoft support,SLAs
Freedom to modify	Full — fork it, patch it, build on it	None — source is locked
Hardware support	Broad, sometimes needs manual setup	Excellentdriversupportoutofbox
Ease of use	Steeper learning curve	GUI-heavy, more accessible

Verdict:

For server deployment, cloud infrastructure, or anywhere we need performance and cost control,Linuxwinsclearly.Zerolicensecost,massivecommunity,andfullcustomizationmakeit hard to beat. The learning curve is steeper but worth it.

Script 1 Hello World Script

Prints a simple 'Hello, World!' message to the terminal. The most basic shell script — used to verify the shell environment is working and understand how echo and script execution work.

```
root@DESKTOP-RJGPDIB:~# echo "Hello world!! Hello Linux"
echo "Hello worldx*b Hello Linux"
Hello worldx*b Hello Linux
root@DESKTOP-RJGPDIB:~#
```

Script 2 AddingTwo Numbers

```
root@DESKTOP-RJGPDIB: /

  GNU nano 7.2
#!/bin/bash

read -p "Enter first number: " num1
read -p "Enter second number: " num2


if ! [[ "$num1" =~ ^-?[0-9]+$ && "$num2" =~ ^-?
    echo "Error: Please enter valid integers."
    exit 1
fi

sum=$((num1 + num2))
echo "Sum: $sum"
```

```
root@DESKTOP-RJGPDIB:/# ./script1.sh
Enter first number: 45
Enter second number: 12
Sum: 57
root@DESKTOP-RJGPDIB:/#
```

Takestwonumbersasinputfromtheuserandpriintstheirsum.Demonstrateshowtorea duser input with the read command, store values in variables, and perform arithmetic using $(( )).

Script 3 Check if a File Exists

```
  GNU nano 7.2                                          script2.sh
#!/bin/bash

read -p "Enter file name: " filename
if [ -f "$filename" ]; then
    echo "File '$filename' exists."
else
    echo "File '$filename' does not exist."
fi
```

```
root@DESKTOP-RJGPDIB:~# nano script2.sh
root@DESKTOP-RJGPDIB:~# chmod +x ./script2.sh
root@DESKTOP-RJGPDIB:~# ./script2.sh
Enter file name: script1.sh
File 'script1.sh' does not exist.
root@DESKTOP-RJGPDIB:~# ls
script2.sh  sumoftwonums.c  sumoftwonums.x
root@DESKTOP-RJGPDIB:~# ./script2.sh
Enter file name: sumoftwonums.c
File 'sumoftwonums.c' exists.
root@DESKTOP-RJGPDIB:~#
```

Accepts a filename as input and checks whether that file exists on the system. Uses an if-then-else condition with the -f flag to test for file existence and prints an appropriate message either way.

Script 4 Print Current Date and Time

```
root@DESKTOP-RJGPDIB: ~
  GNU nano 7.2                                                    scr
#!/bin/bash
echo "current date and time: ($date)"
```

```
root@DESKTOP-RJGPDIB:~# nano script3.sh
root@DESKTOP-RJGPDIB:~# chmod +x script3.sh
root@DESKTOP-RJGPDIB:~# ./script3.sh
current date and time: Sun Mar 29 13:13:04 UTC 2026
root@DESKTOP-RJGPDIB:~#
root@DESKTOP-RJGPDIB:~#
```

This program displays the current date and time as per the system which is in sync with global clock.

Script 5 Simple Menu Script

```
root@DESKTOP-RJGPDIB: ~
  GNU nano 7.2                                                    script4.sh *
#!/bin/bash

while true; do
    echo "==============================="
    echo "         MAIN MENU"
    echo "==============================="
    echo "1. Show Current Date and Time"
    echo "2. Show Current Directory"
    echo "3. List Files in Current Directory"
    echo "4. Show Disk Usage"
    echo "5. Exit"
    echo "==============================="

    # Read user choice
    read -p "Enter your choice [1-5]: " choice

    case $choice in
        1)
            echo "Current date and time: $(date)"
            ;;
        2)
            echo "You are in: $(pwd)"
            ;;
        3)
            echo "Files in current directory:"
            ls -l
            ;;
        4)
            echo "Disk usage:"
            df -h
            ;;
        5)
            echo "Exiting... Goodbye!"
            exit 0
            ;;
        *)
            echo "Invalid choice! Please enter a number between 1 and 5."

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E
```

```
    echo "3. List Files in Current Directory"
    echo "4. Show Disk Usage"
    echo "5. Exit"
    echo "==============================="

    # Read user choice
    read -p "Enter your choice [1-5]: " choice

    case $choice in
        1)
            echo "Current date and time: $(date)"
            ;;
        2)
            echo "You are in: $(pwd)"
            ;;
        3)
            echo "Files in current directory:"
            ls -l
            ;;
        4)
            echo "Disk usage:"
            df -h
            ;;
        5)
            echo "Exiting... Goodbye!"
            exit 0
            ;;
        *)
            echo "Invalid choice! Please enter a number between 1 and 5."
            ;;
    esac
    echo ""
done

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo
```

```
root@DESKTOP-RJGPDIB:~# nano script4.sh
root@DESKTOP-RJGPDIB:~# chmod +x script4.sh
root@DESKTOP-RJGPDIB:~# ./script4.sh
==============================
        MAIN MENU
==============================
1. Show Current Date and Time
2. Show Current Directory
3. List Files in Current Directory
4. Show Disk Usage
5. Exit
==============================
Enter your choice [1-5]: 1
Current date and time: Sun Mar 29 13:15:57 UTC 2026

==============================
        MAIN MENU
==============================
1. Show Current Date and Time
2. Show Current Directory
3. List Files in Current Directory
4. Show Disk Usage
5. Exit
==============================
Enter your choice [1-5]: 2
You are in: /root

==============================
        MAIN MENU
==============================
1. Show Current Date and Time
```

```
==============================
Enter your choice [1-5]: 1
Current date and time: Sun Mar 29 13:15:57 UTC 2026

==============================
        MAIN MENU
==============================
1. Show Current Date and Time
2. Show Current Directory
3. List Files in Current Directory
4. Show Disk Usage
5. Exit
==============================
Enter your choice [1-5]: 2
You are in: /root

==============================
        MAIN MENU
==============================
1. Show Current Date and Time
2. Show Current Directory
3. List Files in Current Directory
4. Show Disk Usage
5. Exit
==============================
Enter your choice [1-5]: 3
Files in current directory:
total 12
-rwxr-xr-x 1 root root 166 Mar 29 13:05 script2.sh
-rwxr-xr-x 1 root root  50 Mar 29 13:12 script3.sh
-rwxr-xr-x 1 root root 964 Mar 29 13:15 script4.sh
-rwxr-xr-x 1 root root   0 Mar 28 18:21 sumoftwonums.c
-rw-r--r-- 1 root root   0 Mar 28 18:21 sumoftwonums.x

==============================
        MAIN MENU
==============================
1. Show Current Date and Time
2. Show Current Directory
3. List Files in Current Directory
4. Show Disk Usage
```

```
==============================
        MAIN MENU
==============================
1. Show Current Date and Time
2. Show Current Directory
3. List Files in Current Directory
4. Show Disk Usage
5. Exit
==============================
Enter your choice [1-5]: 4
Disk usage:
Filesystem      Size  Used Avail Use% Mounted on
none            2.9G     0  2.9G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            2.9G  4.0K  2.9G   1% /mnt/wsl
drivers         472G  212G  260G  45% /usr/lib/wsl/drivers
/dev/sdd       1007G  1.6G  955G   1% /
none            2.9G   76K  2.9G   1% /mnt/wslg
none            2.9G     0  2.9G   0% /usr/lib/wsl/lib
rootfs          2.9G  2.7M  2.9G   1% /init
none            2.9G  524K  2.9G   1% /run
none            2.9G     0  2.9G   0% /run/lock
none            2.9G     0  2.9G   0% /run/shm
none            2.9G   76K  2.9G   1% /mnt/wslg/versions.txt
none            2.9G   76K  2.9G   1% /mnt/wslg/doc
C:\             472G  212G  260G  45% /mnt/c
D:\             5.6G  637M  5.0G  12% /mnt/d
tmpfs           578M   20K  578M   1% /run/user/0

==============================
        MAIN MENU
==============================
1. Show Current Date and Time
2. Show Current Directory
3. List Files in Current Directory
4. Show Disk Usage
5. Exit
==============================
Enter your choice [1-5]: 5
Exiting... Goodbye!
root@DESKTOP-RJGPDIB:~#
```

This script code represents a menu driving program In linux shell displaying contents of the directory files their allocated size etc.